

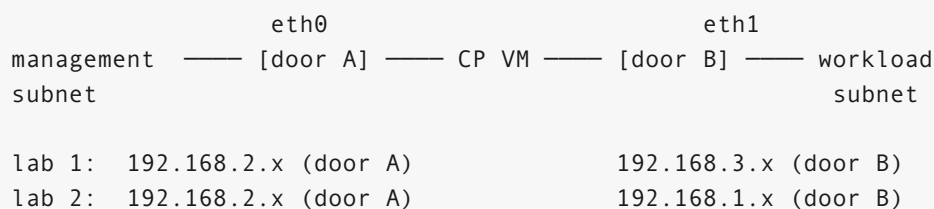
rp_filter and the Supervisor Control Plane — Explained

One Linux kernel rule caused the two most confusing outages in this project — in **lab 1** it stopped the ESXi hosts from joining the Supervisor as nodes; in **lab 2** it stopped vCenter itself from configuring the cluster. Same disease, different patient. This document builds the mechanism up from scratch so the next occurrence costs one log line instead of one evening.

(Symptom→fix recipes live in `TROUBLESHOOTING.md` — “Supervisor ESXi nodes never join” and “vCenter→CP traffic dropped by rp_filter”. This is the *why* behind both.)

1. The setup: the CP VM is a house with two doors

Every Supervisor control-plane VM straddles both subnets:



- **eth0 (door A)** faces the *management* subnet: talking to vCenter, DNS, and receiving vCenter’s configuration pushes.
- **eth1 (door B)** faces the *workload* subnet: kubelet traffic, pods, and serving as the load balancer’s backend.

This dual-homing is required (the Supervisor wizard insists management ≠ workload — see runbook Root Cause #6), and it is precisely what arms the trap below.

2. The rule: reverse-path filtering

When a packet arrives at a Linux machine, the kernel runs an anti-spoofing check called **reverse-path filtering** (`rp_filter`). In **strict** mode — `rp_filter=1`, the default on the Photon-based CP VMs — the question is:

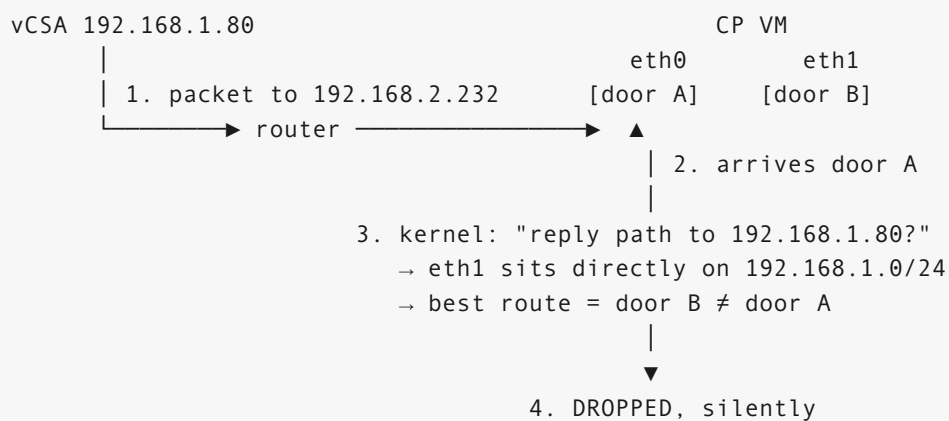
“This packet claims to be from sender X and arrived through door A. If I were sending a packet to X, which door would I use? If the answer is not door A, this looks spoofed — drop it.”

Three properties make this brutal to debug:

1. **The drop is silent.** No error, no log, no ICMP. The sender just times out.
2. **The receiver looks healthy.** Sockets are listening, services are up; packets die in the kernel before anything user-visible.
3. **It's direction-sensitive.** Connections *initiated* by the CP VM work perfectly; only certain *inbound* connections die — so half of an integration works and half doesn't, which reads as “flaky”, not “broken”.

3. The trap, step by step (lab 2's version)

Lab 2's vCenter lives at 192.168.1.80 — on the **workload** subnet, door B's side. But vCenter always addresses the CP VM's **management** IP (192.168.2.232, door A):



vCenter waits out its timeout and logs (in `wcpsvc.log`):

```
Get "http://localhost:1080/external-cert/http1/192.168.2.232/6443/version":
context deadline exceeded
```

Meanwhile **CP-initiated** traffic to that same vCenter goes *out* door B directly and returns to door B — symmetric, flawless. Hence lab 2's baffling half-working state: CSI could log *into* vCenter, but vCenter couldn't write the `AvailabilityZone` CR *into* the cluster — so CSI crashlooped on `could not find any AvailabilityZone`, the CNS CRDs never registered, and vmop failed on `CnsNodeVmAttachment`.

4. Lab 1's version: same rule, different victim

Lab 1's vCSA sits on the **management** subnet (192.168.2.80) — same side as door A, so vCenter↔CP is symmetric and was never a problem.

Lab 1’s victims were the **ESXi hosts**: originally they only had workload-subnet addresses (192.168.3.24x), and spherelet (the ESXi kubelet) must reach the CP’s floating management IP (192.168.2.231, door A). Same walk as above with the labels changed: arrive door A, reverse route says door B, dropped. Spherelet logged `dial tcp 192.168.2.231:6443: i/o timeout` forever and no ESXi node ever joined the cluster (“No node is accepting vSphere Pods”).

5. Side-by-side

	Lab 1	Lab 2
vCSA lives on	management subnet (.2.80)	workload subnet (.1.80)
vCenter → CP door A	same subnet — symmetric, fine	cross-subnet — dropped
ESXi hosts → CP door A	workload-only hosts — dropped	hosts have mgmt vmks — fine
Visible failure	ESXi nodes never join; spherelet i/o timeouts	AvailabilityZone CR missing; CSI/vmop crashloop; WCP <code>context deadline exceeded</code>
Durable fix	vmkernel NIC per host on the mgmt subnet (now in the network module: <code>sup-host-mgmt + nested_host_mgmt_ips</code>)	none in Terraform — optionally a 2nd vCSA NIC on the mgmt subnet

The general law: *any machine that talks to a CP VM address on one subnet, while the CP’s route back to that machine points out the other NIC, is silently dropped under strict `rp_filter`.* Before enabling a Supervisor in a new environment, check where the vCSA and the ESXi hosts sit relative to the CP’s two doors.

6. The fix, and what it actually does

On **every CP VM** (root password via `decryptK8Pwd.py` on the vCSA):

```
sysctl -w net.ipv4.conf.all.rp_filter=2 -w net.ipv4.conf.eth0.rp_filter=2
```

Mode `2` is **loose** reverse-path filtering: the question relaxes to “do I have *any* route back to this sender?” — designed by the kernel developers for exactly this situation (multi-homed hosts with legitimate asymmetric paths). It is not `0` (off); genuinely unroutable spoofed sources are still dropped.

Verification is instant — the previously-dropped path answers immediately (e.g. from the vCSA:

```
curl -sk --max-time 5 https://<cp-mgmt-ip>:6443/version , or from a host: vmkping <cp-floating-ip> ).
```

⚠ Volatile. *The sysctl lives in kernel memory of VMware-managed VMs. Every control-plane redeploy (upgrade, HA repair, re-enable) resets it to strict, and the environment's characteristic failure signature returns. Re-applying takes two minutes — recognizing it is the hard part, which is why this document exists.*

7. How to recognize it in under a minute

1. **Smell:** something that talks to the Supervisor control plane times out, while the reverse direction demonstrably works.
2. **Confirm the drop:** from the affected machine, ping/curl the CP management IP — dead. From a machine on the management subnet — alive. That asymmetry is the tell.
3. **Confirm the cause on the CP VM:**

```
ip route get <sender-ip>          # → dev eth1    (reply exits door B)
sysctl net.ipv4.conf.eth0.rp_filter # → 1        (strict)
```

Arrived door A + reply-route door B + strict = case closed.